ToyRental Platform

Business & Technical Documentation

Navi Mumbai · Toy Rental Marketplace

Full Documentation Package

Project Charter · Business Case · Product Requirements · Technical Design

Version 1.0 · August 2026

Prepared by: Hardik Patil · Performance Test Engineer

Table of Contents

1. Project Charter

1.1 Executive Summary

ToyRental Platform is a microservices-based digital marketplace enabling parents in Navi Mumbai to rent premium kids toys at a fraction of the purchase cost. The platform addresses a genuine consumer pain point: expensive toys that children outgrow within weeks. Built with production-grade engineering practices including event-driven architecture, real-time availability tracking, and comprehensive observability, this platform serves as both a viable business concept and a portfolio showcase for advanced performance engineering and Kubernetes management skills.

> **Portfolio Purpose:** This project is designed as a learning vehicle for performance engineering, Kubernetes cluster management, and bottleneck identification using JMeter, Prometheus, Grafana, and Dynatrace — mirroring real-world enterprise engineering practices from production BSS/telecom environments.

1.2 Project Objectives

Objective Description Success Metric		
Functional Platform	Build a working toy rental marketplace with booking, payment and notifications	All 9 sprints completed, app runs end-to-end
Perf Engineering	Identify and fix minimum 6 production-grade bottlenecks under load	Before/after Grafana proof for each fix
K8s Mastery	Deploy and manage all services on Kubernetes with HPA, PVC, NetworkPolicy	HPA auto-scales during JMeter load tests
Observability	Full observability stack — Prometheus, Grafana, Zipkin with 7 dashboards	All services visible in Grafana with custom metrics

1.3 Scope

In Scope — MVP (v1.0):

- Toy catalogue with search and availability calendar
- Customer registration, login and profile management
- Booking flow with date selection and UPI payment (mocked via WireMock)
- Real-time availability tracking via Couchbase snapshots
- Kafka-driven async notifications (WhatsApp — mocked via WireMock)

- Monthly report generation as PDF stored in MinIO
- Admin dashboard — deliveries, pickups, overdue tracking
- Full Kubernetes deployment with Helm charts
- Performance testing with 7 JMeter test plans

Out of Scope — v1.0:

- Real payment gateway integration (Razorpay live)
- Real WhatsApp Business API integration
- Mobile app (iOS / Android)
- Multi-city expansion beyond Navi Mumbai
- Recommendation engine or ML features

1.4 Stakeholders

Stakeholder Role Interest		
Hardik Patil	Product Owner + Developer	Business launch + portfolio showcase
Parents (Navi Mumbai)	End Users	Affordable toy access for children
Interviewers / Hiring Managers	Portfolio Audience	Evidence of production-grade engineering skills

1.5 Timeline & Milestones

Sprint Milestone Duration		
Sprint 1	Infrastructure setup — Docker Compose, Maven, Gateway, Flyway, WireMock	1 week
Sprint 2	Toy Service — catalogue, availability, Couchbase, logical date	1 week
Sprint 3	Booking Service — bookings, payments, admin APIs	1 week
Sprint 4	Kafka event pipeline — all topics, DLT, idempotency	1 week
Sprint 5	Month-end report — PDF generation, MinIO, Couchbase report doc	1 week
Sprint 6	Observability — Prometheus, Grafana dashboards, Zipkin	1 week
Sprint 7	Performance engineering — 7 JMeter test plans, bottleneck fixes	1 week
Sprint 8	Kubernetes + Helm — HPA, PVC, NetworkPolicy, NGINX Ingress	1 week

Sprint Milestone Duration		
Sprint 9	React frontend — catalogue, booking flow, admin dashboard	1 week

2. Business Case

2.1 Problem Statement

Premium kids toys in India — LEGO sets, Fisher-Price activity centres, science kits, outdoor ride-ons — cost between ■2,000 and ■15,000. Children typically lose interest within 3-4 weeks of receiving a toy, leaving parents with an expensive, unused item and no practical way to recover the cost. The toy resale market in India is fragmented and unreliable, and parents in urban areas like Navi Mumbai lack access to a trusted, convenient toy rental service.

■6,500	3-4 wks	85%	1.1M
Avg toy price (premium)	Child interest span	Weekly rental saving	Target city population

2.2 Proposed Solution

ToyRental Platform enables parents to rent premium toys on weekly or monthly basis with free doorstep delivery and pickup within Navi Mumbai. A fully digital experience — browse catalogue, check real-time availability, book online, pay via UPI — removes all friction from the rental process. A refundable deposit protects inventory. An admin dashboard gives the business owner full visibility into deliveries, returns, and revenue.

> **Core Value Proposition:** Rent a ■6,500 LEGO set for ■499/week instead of buying it. Save ■6,000. Child gets a new toy every month. No clutter. No waste.

2.3 Market Opportunity

Navi Mumbai has a population of approximately 1.1 million with a significant upper-middle-class demographic concentrated in areas like Vashi, Kharghar, Nerul, and Belapur. These areas have high smartphone penetration and strong UPI adoption — ideal conditions for a digital-first rental service.

Segment	Estimate	Assumption
Families with children aged 2-12	~180,000 families	~16% of Navi Mumbai population
Early adopters (tech-savvy, UPI users)	~18,000 families	10% of target segment
Reachable via Instagram (Year 1)	~500-1,000 customers	5% conversion from 10,000 ad reach

2.4 Revenue Model

Revenue is generated through weekly and monthly rental fees plus deposit income (float). Secondary revenue comes from late return fees and optional toy sanitization premium.

Tier Toy MRP Range Weekly Rent Monthly Rent Deposit				
Budget	■1,000 – ■3,000	■199 – ■299	■599 – ■799	■500 – ■800
Mid	■3,000 – ■7,000	■399 – ■599	■999 – ■1,499	■1,000 – ■2,000
Premium	■7,000+	■699 – ■999	■1,799 – ■2,499	■2,000 – ■3,500

2.5 Risks & Mitigation

Risk Likelihood Impact Mitigation			
Toy damaged by customer	High	Medium	Deposit collected upfront. Photo evidence before/after. Partial refund policy.
Customer does not return toy	Low	High	Deposit covers cost. WhatsApp reminders on overdue.
Low initial demand	Medium	High	Validate with WhatsApp/Instagram before tech build. Manual process first.
Hygiene concerns from parents	Medium	Medium	Sanitize every toy. Mention in every listing and Instagram post.
Tech downtime during booking	Low	Medium	K8s HPA + readiness probes + circuit breaker on payment.

3. Product Requirements Document (PRD)

3.1 User Personas

Priya, 32 — Working Mother	Rajan, 38 — Stay-at-Home Dad
Lives in Vashi. Has a 4-year-old. Buys premium toys but child loses interest fast. Wants affordable, hygienic toys. Browses on phone. Pays via UPI. Expects WhatsApp confirmation.	Lives in Kharghar. Has two kids aged 6 and 9. Manages household budget carefully. Wants variety — different toy every month. Values easy return pickup. Checks reviews before renting.

3.2 Core Features — MVP

Feature	Description	Priority
Toy Catalogue	Browse all toys with photos, age group, category filters, availability status	**Must Have**
Availability Calendar	View blocked dates per toy. Check if toy is available for selected dates	**Must Have**
Booking Flow	Select toy + dates + delivery address. Confirm booking. Pay deposit + rent	**Must Have**
UPI Payment	Pay via UPI (mocked via WireMock in dev). COD option available	**Must Have**
WhatsApp Notification	Booking confirmation, delivery reminder, overdue alert via WhatsApp (mocked)	**Must Have**
My Bookings	View active and past bookings. See status, return date, deposit status	**Must Have**
Admin Dashboard	Today's deliveries, pickups, overdue list, revenue summary	**Must Have**
Monthly PDF Report	Auto-generated month-end report — revenue, top toys, settlements	**Must Have**
Toy Image Upload	Admin uploads multiple photos per toy. Primary photo shown in catalogue	**Should Have**
Booking Extension	Customer or admin extends rental period. Price recalculated	**Should Have**
Late Return Fee	System flags overdue. Admin can charge late fee before refunding deposit	**Could Have**

3.3 User Stories

Epic: Toy Discovery

- As a parent, I want to browse all available toys so that I can find the right toy for my child's age group.
- As a parent, I want to filter toys by category and age so that I don't waste time looking at irrelevant items.
- As a parent, I want to see a toy's availability calendar so that I know if it's free for the dates I need.

Epic: Booking

- As a parent, I want to book a toy for a specific date range so that I can plan delivery around my schedule.
- As a parent, I want to pay via UPI so that the process is fast and familiar.
- As a parent, I want to receive a WhatsApp confirmation immediately after booking so that I have proof of my booking.
- As a parent, I want to extend my rental period online so that I don't have to call anyone.

Epic: Admin Operations

- As an admin, I want to see today's deliveries and pickups so that I can plan my day.
- As an admin, I want to see overdue returns so that I can follow up with customers.
- As an admin, I want to mark a toy as returned and note its condition so that I can update inventory accurately.
- As an admin, I want a monthly PDF report so that I can track revenue and popular toys.

3.4 Non-Functional Requirements

Category Requirement Target		
Performance	API response time under normal load (100 users)	p99 < 500ms
Performance	Toy availability check (Couchbase read)	p99 < 50ms
Performance	Booking creation end-to-end	p99 < 2000ms
Scalability	Concurrent users supported without degradation	500 users
Reliability	Service uptime during business hours	99.5%
Reliability	Zero double-bookings under concurrent load	0 oversells
Recovery	Service restart recovery time (K8s readiness probe)	< 60 seconds
Security	All endpoints protected by JWT	100% coverage
Observability	All services emitting Prometheus metrics	100% coverage
Kafka	Notification delivery lag under normal load	< 5 seconds

4. Technical Design Document

4.1 Architecture Overview

The platform uses a microservices architecture with 2 core domain services behind an API Gateway. Services communicate asynchronously via Apache Kafka for all non-critical paths. Real-time toy availability is cached in Couchbase for sub-millisecond reads. All services are deployed on Kubernetes (Docker Desktop locally) with Helm charts managing environment-specific configuration.

Key Design Decisions: (1) Couchbase for availability — avoids complex JOIN queries on booking table at read time. (2) Kafka for booking events — decouples availability update and notification from the booking transaction. (3) WireMock for external APIs — enables realistic integration testing without live Razorpay or WhatsApp accounts. (4) Logical date in Couchbase — enables controlled date simulation for month-end and overdue testing.

4.2 Microservices Design

Service Port Responsibility DB Kafka Role				
API Gateway	8080	Routing, JWT validation, rate limiting (Redis), circuit breaker, correlation ID injection	Redis	None
Toy Service	8081	Toy catalogue, search, availability calendar, Couchbase snapshot management	PostgreSQL + Couchbase	Consumer: booking.confirmed, booking.cancelled
Booking Service	8082	Bookings, payments (WireMock), notifications (WireMock), monthly PDF report	PostgreSQL + Couchbase	Producer: all topics. Consumer: payment.*, month.end.trigger

4.3 Database Schema

Toy Service — PostgreSQL

Table Key Columns Purpose		
toys	id, name, category, age_group, weekly_price, monthly_price, deposit_amount, status, condition	Master toy catalogue
toy_images	id, toy_id, url, is_primary, sort_order	Multiple photos per toy
toy_availability_log	id, toy_id, booking_id, blocked_from, blocked_to, action, reason	Audit trail for Couchbase rebuild

Booking Service — PostgreSQL

Table Key Columns Purpose		
customers	id, name, phone, email, password_hash, area, address, pincode	Customer accounts
bookings	id, toy_id, customer_id, start_date, end_date, rental_type, rental_amount, deposit_amount, status, payment_status	Rental bookings
payments	id, booking_id, amount, type, method, status, razorpay_order_id, razorpay_payment_id	Payment records
notifications	id, booking_id, customer_id, type, channel, message, status, sent_at	Notification log
monthly_reports	id, month, year, total_bookings, total_revenue, top_toy_id, pdf_storage_path, status	Report metadata

Couchbase Buckets

Bucket Document ID Pattern Purpose		
toy-availability	avail::toy-{id}	Real-time blocked dates snapshot per toy. Updated via Kafka consumer.
monthly-reports	report::yyyy-mm	Pre-computed report summary for fast admin reads. PDF path stored here.
logical-date	logical-date::current	Centralized business date. Enables date simulation for testing.

4.4 Kafka Event Design

Topic Publisher Consumer(s) Partitions Key				
booking.confirmed	Booking Service	Toy Service, Notification consumer	6	toyId
booking.cancelled	Booking Service	Toy Service, Notification consumer	6	toyId
booking.overdue	Booking Service	Notification consumer	3	customerId
payment.success	Booking Service	Booking Service (internal)	6	bookingId
payment.failed	Booking Service	Booking Service (internal)	6	bookingId
month.end.trigger	Admin API	Booking Service	1	month-year
monthly.report.generated	Booking Service	Future: email service	1	month-year

Idempotency: Every Kafka consumer checks eventId before processing. Each topic has a Dead Letter Topic (DLT) for failed events. Correlation ID flows from HTTP header through Kafka message header into logs.

4.5 API Endpoints

Toy Service — Port 8081

Method Endpoint Description		
GET	`/api/v1/toys`	Paginated toy catalogue with filters
GET	`/api/v1/toys/{id}`	Toy detail
GET	`/api/v1/toys/search?q=&category;=&age;=`	Search toys
GET	`/api/v1/toys/{id}/availability?from=&to;=`	Check availability for date range
GET	`/api/v1/toys/available?from=&to;=`	Browse only available toys
POST	`/api/v1/toys`	Add new toy (admin)
PUT	`/api/v1/toys/{id}`	Update toy (admin)

Method Endpoint Description		
GET	`/api/v1/admin/toys/inventory`	Full inventory status (admin)

Booking Service — Port 8082

Method Endpoint Description		
POST	`/api/v1/customers/register`	Register new customer
POST	`/api/v1/customers/login`	Login — returns JWT
POST	`/api/v1/bookings`	Create booking + initiate payment
GET	`/api/v1/bookings/{id}`	Booking detail
PUT	`/api/v1/bookings/{id}/cancel`	Cancel booking
PUT	`/api/v1/bookings/{id}/extend`	Extend rental period
POST	`/api/v1/payments/initiate`	Initiate UPI payment (WireMock)
POST	`/api/v1/payments/webhook`	Razorpay webhook (WireMock callback)
POST	`/api/v1/admin/reports/trigger`	Trigger month-end report via Kafka
GET	`/api/v1/admin/reports/{id}/pdf`	Download PDF from MinIO
GET	`/api/v1/admin/bookings/today/deliveries`	Today's deliveries
GET	`/api/v1/admin/bookings/overdue`	Overdue returns

4.6 Infrastructure & Deployment

All services are deployed on Docker Desktop Kubernetes locally. Helm charts manage environment-specific configuration via values-dev.yaml and values-prod.yaml. Three namespaces isolate application, infrastructure, and monitoring concerns.

Component Type Namespace K8s Features			
api-gateway	Deployment	toy-rental	HPA, ConfigMap, Secret, Ingress
toy-service	Deployment	toy-rental	HPA (CPU 60%), Liveness + Readiness probes, PDB
booking-service	Deployment	toy-rental	HPA (CPU 60%), Liveness + Readiness probes, PDB min 2
PostgreSQL	StatefulSet	infra	PVC 10GB, Secret for credentials
Couchbase	StatefulSet	infra	PVC 5GB, ConfigMap for buckets
Kafka + Zookeeper	StatefulSet	infra	PVC 10GB, 6 partitions per topic
Redis	Deployment	infra	ConfigMap for TTL config
MinIO	Deployment	infra	PVC 20GB, S3-compatible API
WireMock	Deployment	infra	ConfigMap for stub mappings
Keycloak	Deployment	infra	ConfigMap for realm, Ingress for console
Prometheus	Deployment	monitoring	ConfigMap for scrape config, PVC 5GB
Grafana	Deployment	monitoring	ConfigMap for dashboards, PVC 2GB
Zipkin	Deployment	monitoring	In-memory storage

4.7 Performance Engineering Plan

Seven JMeter test plans cover all critical performance scenarios. Each test targets a specific bottleneck, with before/after proof documented via Grafana screenshots.

Test Plan Scenario Bottleneck to Find Fix			
Catalogue Browse	500 concurrent users browsing	Missing index on category + age_group	Composite index

Test Plan Scenario Bottleneck to Find Fix			
Concurrent Booking	200 users booking same toy	Double booking race condition	SELECT FOR UPDATE
Cache Stampede	Couchbase restart + 500 users	All cache miss → DB overwhelmed	Cache warming on startup
Payment Retry Storm	WireMock 503 at 30% rate	Retry flood + circuit breaker open	Resilience4j CB
Kafka Consumer Lag	1000 rapid bookings	Notification 60s delayed	Increase partitions
Soak Test	100 users for 4 hours	JVM heap grows unbounded	JFR + G1GC tuning
Spike Test	50→2000→50 in 30s	HPA reaction time + pod startup	Pre-scale + readiness probe

4.8 Observability Strategy

Tool Purpose Key Dashboards / Metrics		
Prometheus	Metrics collection — scrapes all /actuator/prometheus endpoints	RPS, latency p50/p95/p99, error rate, JVM heap, DB pool
Grafana	Visualization — 7 dashboards covering all services and infra	Services Overview, Kafka Lag, JVM Health, Couchbase Cache, K8s Scaling, Booking Conflicts, Business Metrics
Zipkin	Distributed tracing — full trace for booking flow across services	Trace: gateway → toy-service → booking-service → Kafka
Structured Logs	JSON logs with correlationId, traceId, spanId in all services	Correlate a single booking request across all service logs
Custom Metrics	Micrometer counters/timers for business events	booking.created.total, availability.cache.hit.ratio, payment.success.rate, pdf.generation.duration

5. Agile Delivery Plan

5.1 Sprint Plan

Sprint Epic Key Deliverables Stories Points				
S1	**Infrastructure**	Docker Compose, Maven multi-module, API Gateway, Flyway, WireMock stubs, Keycloak	6	**21**
S2	**Toy Service**	Toy CRUD, catalogue API, Couchbase availability, logical date bucket	9	**34**
S3	**Booking Service**	Customer auth, booking flow, payment (WireMock), admin APIs	10	**42**
S4	**Kafka Pipeline**	All topics, DLT, consumer groups, idempotency, correlation ID	10	**34**
S5	**Month-End Report**	Kafka trigger, PDF generation (iText), MinIO upload, Couchbase report doc	8	**21**
S6	**Observability**	Prometheus, Grafana 7 dashboards, Zipkin, custom metrics	7	**21**
S7	**Performance Eng.**	7 JMeter test plans, 6 bottleneck finds + fixes, Grafana proof	8	**34**
S8	**Kubernetes**	Helm charts, HPA, PVC, NetworkPolicy, PDB, NGINX Ingress	7	**21**
S9	**React Frontend**	Catalogue, booking flow, admin dashboard, mobile responsive	6	**21**
TOTAL			**71**	**249**

5.2 Definition of Done

A story is considered done when ALL of the following are true:

- Code compiles and all unit tests pass

- Integration tests pass (WireMock stubs used for external calls)
- API returns correct response verified via Postman / automated test
- Flyway migration applied cleanly on fresh DB
- Service starts successfully in Docker Compose
- Prometheus metrics visible in Grafana for this service
- Zipkin shows correct trace for the flow
- K8s Deployment + Service YAML applied successfully via Helm
- Readiness probe returns healthy within 60 seconds of pod start
- No hardcoded secrets — all config via ConfigMap / Secret / values.yaml

---

Document prepared by Hardik Patil · Performance Test Engineer · August 2026 · Version 1.0